PoC: Wazuh file-integrity alerting with Discord notifications

Oct 12, 2026 · @MJ

Objective and result

This PoC shows that a Wazuh server can detect file changes on a Windows endpoint, raise them as high-severity alerts using custom rules, and push them to a Discord channel within seconds. The test passed end to end on 2026-10-11.

- **Goal:** detect creation, modification and deletion of files in a watched folder and notify an analyst without opening the dashboard.
- **Result:** the Windows agent reported all three events, custom rules 100100, 100101 and 100102 raised them to level 12, 12 and 14, and each alert arrived in Discord with the file path and event type.
- **Scope:** a home lab built for learning, as groundwork for the Achieve Without Borders SIEM project. It is not a production deployment.

Lab environment

Two machines, one Wazuh 4.14.8 deployment, all on the home network.

Component	Role	Details
Kali Linux VM (VirtualBox)	Wazuh server	All-in-one install: manager, indexer, dashboard and Filebeat; address 192.168.1.9
Windows 11 Pro PC (WindowsMJ)	Monitored endpoint	Wazuh agent 4.14.8, agent ID 001, address 192.168.1.2
Discord channel	Alert destination	Reached through a channel webhook
Watched folder	Test target	C:\FIMTest on the Windows PC, monitored in realtime

Implementation

Four changes turn a stock install into this alerting pipeline.

1. **Install the server.** Run the official all-in-one installer on Kali (`wazuh-install.sh -a`), then enrol the Windows PC from the dashboard's Deploy new agent page. The agent shows as Active in Endpoints.
2. **Watch a folder on the agent.** In the Windows agent's `ossec.conf`, inside `<syscheck>`, add a realtime directory entry, then restart the agent service:

```
<directories realtime="yes" check_all="yes"
report_changes="yes">C:\FIMTest</directories>
```

3. **Raise severity with custom rules.** On the server, add these rules to `/var/ossec/etc/rules/local_rules.xml`. Each one matches a built-in file integrity rule (554 added, 550 modified, 553 deleted) when the path contains FIMTest:

```
<group name="syscheck,custom,">
  <rule id="100100" level="12">
    <if_sid>554</if_sid>
    <field name="file" type="pcre2">(?i)fimtest</field>
    <description>ALERT: New file created in watched folder
C:\FIMTest</description>
  </rule>
  <rule id="100101" level="12">
    <if_sid>550</if_sid>
    <field name="file" type="pcre2">(?i)fimtest</field>
    <description>ALERT: File modified in watched folder
C:\FIMTest</description>
  </rule>
  <rule id="100102" level="14">
    <if_sid>553</if_sid>
    <field name="file" type="pcre2">(?i)fimtest</field>
    <description>CRITICAL: File deleted from watched folder
C:\FIMTest</description>
  </rule>
</group>
```

4. **Send level 12 and above to Discord.** Create a channel webhook, add a small custom integration script on the server, and register it in the server's `ossec.conf`. The webhook address is a secret and is left out here:

```
<integration>
  <name>custom-discord</name>
  <hook_url>[DISCORD WEBHOOK URL]</hook_url>
  <level>12</level>
  <alert_format>json</alert_format>
</integration>
```

The script lives in `/var/ossec/integrations/` as `custom-discord` plus `custom-discord.py`, owned by root:wazuh with mode 750. It posts the rule description, agent, rule ID and level, file path, event type and timestamp as a Discord embed, orange for level 12 and red for level 14 and above. After each config change, check with `wazuh-analysisd -t` and restart the manager.

Test results

All three file events were detected, raised to the new severity and delivered to Discord. The test created a file in C:\FIMTest, appended a line to it, then deleted it, with a 5 second pause between steps.

Test	Built-in alert (before)	Custom alert (after)	Discord
Create file	Rule 554, level 5	Rule 100100, level 12	Orange alert received
Modify file	Rule 550, level 7	Rule 100101, level 12	Orange alert with before and after hashes
Delete file	Rule 553, level 7	Rule 100102, level 14	Red CRITICAL alert received

The dashboard's Last 24 hours panel counted 3 alerts in the High severity band (levels 12 to 14) after the test, and 0 in Critical (level 15 or higher), matching the levels configured. The Document Details view of the modify event records the evidence an analyst needs: the file path, the event type, the old and new MD5, SHA1 and SHA256 hashes, and the size change from 16 to 34 bytes.

Evidence to attach

Insert these screenshots under the table, in this order. Blur the webhook address in any screenshot that shows the server's `ossec.conf`.

Figure	Screenshot	What it proves
1	Endpoints page with WindowsMJ Active	The agent is enrolled and connected
2	Windows agent `ossec.conf` showing the FIMTest directories line	The folder is monitored in realtime
3	`local_rules.xml` in nano	The three custom rules exist
4	Events list showing rules 554, 550 and 553	Default alerts before the custom rules
5	Events filtered by rule 100100, then 100101, then 100102	Custom rules fired at levels 12, 12 and 14
6	Document Details of the modify event	Hash and size evidence is captured
7	Overview with High severity at 3	The alerts roll up into the severity counters
8	Discord channel with the orange and red alerts	Notification delivery works

Issues and fixes

Five problems cost most of the build time. Each is worth knowing before the real deployment.

Problem	Cause	Fix
Installer refused to run: Filebeat already installed, then ports 9200, 1514, 1515, 55000 and 443 in use	Leftovers from an earlier Wazuh install on the same VM	Purge the old packages and data directories, stop leftover processes, free the ports, then rerun the installer with the overwrite and ignore-checks options

Problem	Cause	Fix
Dashboard showed a 500 error right after install or restart	The manager's API was still starting	Wait a few minutes, hard refresh the page, and check `api.log` if it persists
Discord received nothing from the built-in Slack integration	Discord answered 400 to Wazuh's Slack-style message, which carries the alert ID in a timestamp field	Replace it with a custom integration script that posts a native Discord embed (HTTP 204 on success)
Dashboard service restarted in a loop with Port 443 already in use	An old dashboard process still held the port	Stop the service, kill the leftover process, confirm the port is free, start once
Agents showed Disconnected and `agent_control` could not reach the Wazuh database socket	The manager was half started after repeated restarts, with duplicate API processes on port 55000	Stop all Wazuh manager processes, then start indexer, manager and dashboard in that order and wait for each

A recurring cause behind several of these was restarting services before the previous start had finished. Waiting 60 to 90 seconds between starts avoided most of the trouble.